

1 Write a Python program to demonstrate polymorphism using functions and classes.

Aim

To write a Python program that demonstrates polymorphism (same function or method name behaving differently depending on input or object type) using both functions and classes.

Algorithm

1. Function-based polymorphism
 - Define a function that behaves differently depending on the type of argument.
 - Example: can add integers, concatenate strings, or merge lists.
2. Class-based polymorphism
 - Create a base class with a method (e.g.,).
 - Create multiple derived classes (e.g., ,) that override the method.
 - Call the method on different objects to show polymorphic behavior.

CODE

```
# Polymorphism using Functions
```

```
def add(x, y):
```

```
    return x + y
```

```
print("Function Polymorphism:")
```

```
print("Add integers:", add(10, 20))
```

```
print("Add strings:", add("Hello ", "World"))
```

```
print("Add lists:", add([1, 2], [3, 4]))
```

```
# Polymorphism using Classes
```

```
class Animal:
```

```
    def speak(self):
```

```
    return "This animal makes a sound."
```

```
class Dog(Animal):  
    def speak(self):  
        return "Dog says: Woof!"
```

```
class Cat(Animal):  
    def speak(self):  
        return "Cat says: Meow!"
```

```
print("\nClass Polymorphism:")  
animals = [Dog(), Cat(), Animal()]
```

```
for a in animals:  
    print(a.speak())
```

OUTPUT

Function Polymorphism:

Add integers: 30

Add strings: Hello World

Add lists: [1, 2, 3, 4]

Class Polymorphism:

Dog says: Woof!

Cat says: Meow!

This animal makes a sound.

Conclusion

- Function polymorphism: The same function works differently depending on argument types (int, str, list).
- Class polymorphism: The same method behaves differently depending on the object (, ,).
- Polymorphism allows flexibility and reusability in code, making it a core concept in object-oriented programming.

2 Write a program to override a parent class method in a child class.

Aim

To write a Python program that shows how a child class overrides a parent class method, allowing the child class to provide its own implementation.

Algorithm

1. Define a parent class with a method (e.g.,).
2. Define a child class that inherits from the parent class.
3. In the child class, override the parent's method by redefining it.
4. Create objects of both classes.
5. Call the method on each object to demonstrate overriding.

CODE

Program to demonstrate method overriding

Step 1: Parent class

class Parent:

def display(self):

print("This is the Parent class method.")

Step 2: Child class overriding the method

class Child(Parent):

```
def display(self):  
    print("This is the Child class method (overridden).")
```

Step 3: Create objects

```
p = Parent()  
c = Child()
```

Step 4: Call methods

```
print("Calling method from Parent object:")  
p.display()
```

```
print("\nCalling method from Child object:")  
c.display()
```

OUTPUT

Calling method from Parent object:
This is the Parent class method.

Calling method from Child object:
This is the Child class method (overridden).

Conclusion

- The child class overrides the parent class method by redefining it.
- When the method is called on a child object, the child's version executes instead of the parent's.
- This demonstrates runtime polymorphism in object-oriented programming.